Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №2

По дисциплине «Тестирование программного обеспечения»

Выполнил:

Студенты группы P3306,

Михайлов Дмитрий Андреевич

Медведев Владислав Александрович

Преподаватель:

Кулинич Ярослав Вадимович

УНИВЕРСИТЕТ ИТМО

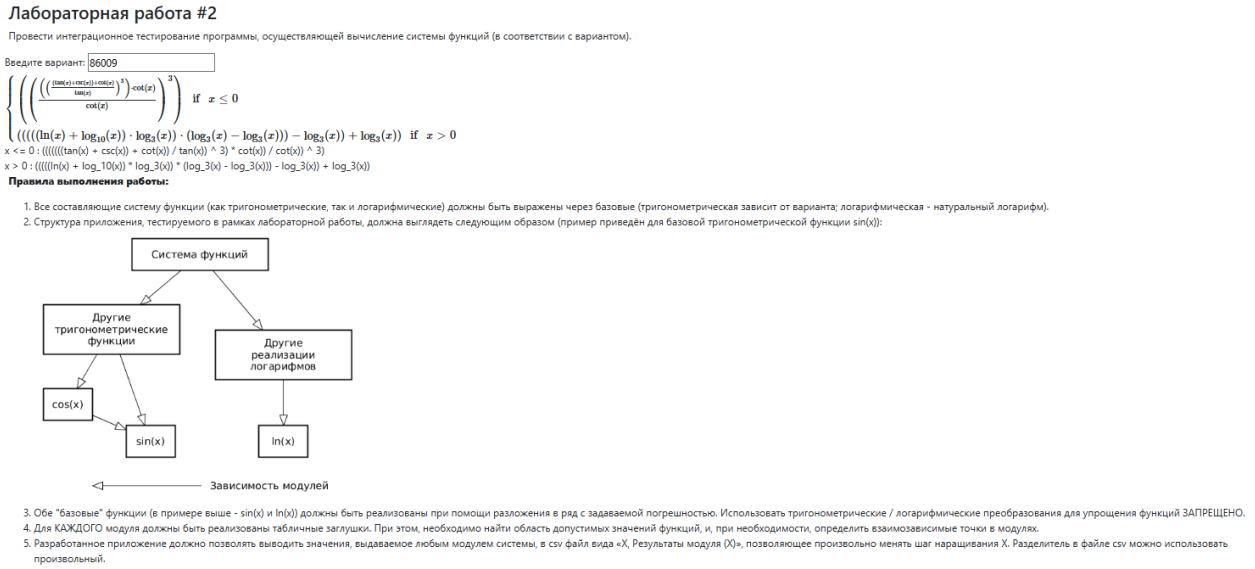
Санкт-Петербург

2026 год

Содержание

Задание 3
UML-диаграмму классов разработанного приложения 3
Описание тестового покрытия с обоснованием его выбора 3
Графики, построенные csv-выгрузкам, полученным в процессе интеграции приложения 6
Вывод по работе 9

Задание



Лабораторная работа #2

Провести интеграционное тестирование программы, осуществляющей вычисление системы функций (в соответствии с вариантом).

Введите вариант: 86009

$$\begin{cases} \left(\left(\left(\frac{\left(\left(\frac{(\tan(x)+\csc(x))+\cot(x)}{\tan(x)}\right)^2\right)\cdot\cot(x)}{\cot(x)}\right)^3\right) & \text{if } x \le 0 \\ (((((\ln(x)+\log_{10}(x))\cdot\log_3(x))\cdot(\log_3(x)-\log_3(x)))-\log_3(x))+\log_3(x)) & \text{if } x > 0 \end{cases}$$

x <= 0 : (((((((tan(x) + csc(x)) + cot(x)) / tan(x)) ^ 3) * cot(x)) / cot(x)) ^ 3)

x > 0 : (((((ln(x) + log_10(x)) * log_3(x)) * (log_3(x) - log_3(x))) - log_3(x)) + log_3(x))

Правила выполнения работы:

1. Все составляющие систему функции (как тригонометрические, так и логарифмические) должны быть выражены через базовые (тригонометрическая зависит от варианта; логарифмическая - натуральный логарифм).
2. Структура приложения, тестируемого в рамках лабораторной работы, должна выглядеть следующим образом (пример приведён для базовой тригонометрической функции sin(x)):

3. Обе "базовые" функции (в примере выше - sin(x) и ln(x)) должны быть реализованы при помощи разложения в ряд с задаваемой погрешностью. Использовать тригонометрические / логарифмические преобразования для упрощения функций ЗАПРЕЩЕНО.
4. Для КАЖДОГО модуля должны быть реализованы табличные заглушки. При этом, необходимо найти область допустимых значений функций, и, при необходимости, определить взаимозависимые точки в модулях.
5. Разработанное приложение должно позволять выводить значения, выдаваемое любым модулем системы, в csv файл вида «X, Результаты модуля (X)», позволяющее произвольно менять шаг наращивания X. Разделитель в файле csv можно использовать произвольный.

Рис. 1. Задание по варианту.

UML-диаграмму классов разработанного приложения

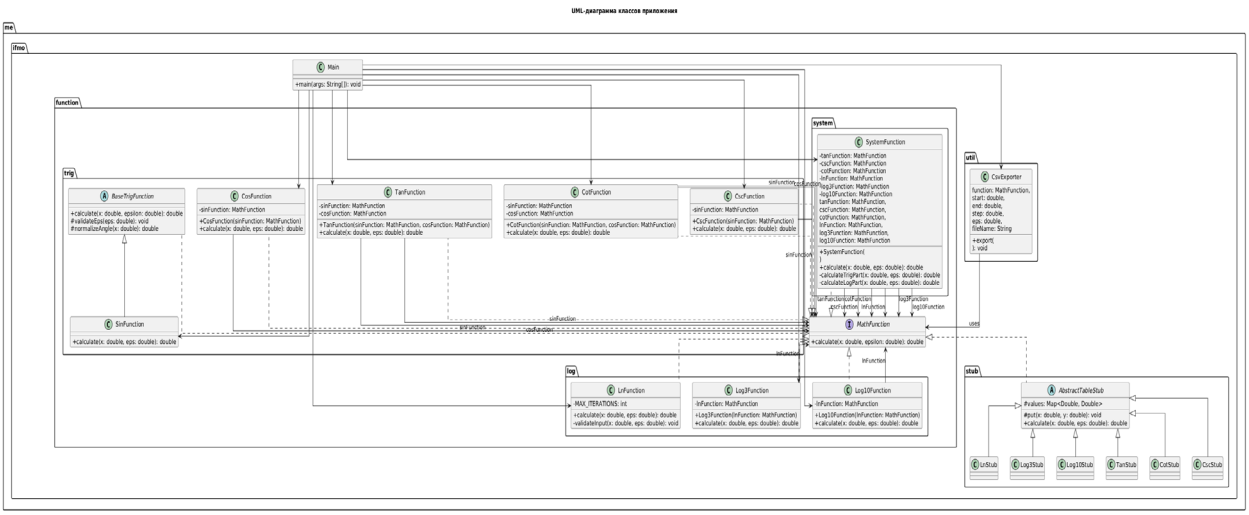


Рис. 2. UML-диаграмма приложения.

Описание тестового покрытия с обоснованием его выбора

Для проверки разработанного приложения было реализовано многоуровневое тестовое покрытие, включающее модульные и интеграционные тесты. В соответствии с заданием тестирование выполнялось с использованием JUnit 5, с учётом анализа эквивалентности входных данных и особенностей области допустимых значений функций.

1. Модульные тесты базовых функций

На первом этапе были протестированы базовые функции системы: SinFunction и LnFunction. Выбор именно этих модулей в качестве начальной точки тестирования обусловлен тем, что они являются нижним уровнем архитектуры приложения и используются при вычислении остальных функций.

Для функции LnFunction были выделены следующие классы эквивалентности:

- корректные положительные аргументы x > 0;
- значения, близкие к единице;
- малые положительные значения;
- стандартные положительные значения;
- недопустимые значения x <= 0;
- некорректные значения точности eps <= 0.

Для функции SinFunction были выделены следующие классы эквивалентности:

- нулевое значение аргумента;
- стандартные положительные углы;
- стандартные отрицательные углы;
- точки, в которых значение функции известно точно;
- значения, требующие нормализации угла;
- некорректные значения точности eps <= 0.

Такое покрытие позволяет проверить не только корректность вычислений, но и корректность обработки граничных случаев и ошибочных входных данных.

2. Модульные тесты производных функций

После проверки базовых функций были разработаны модульные тесты для производных модулей:
Log3Function, Log10Function, CosFunction, TanFunction, CotFunction, CscFunction.

Для этих классов использовался изолированный подход к тестированию: зависимости подменялись простыми заглушками, реализующими интерфейс MathFunction. Это позволило проверять именно внутреннюю логику конкретного модуля, не смешивая её с возможными погрешностями или ошибками базовых функций.

Например:

- для Log3Function и Log10Function проверялось правильное вычисление отношения значений натурального логарифма;
- для TanFunction проверялось корректное деление sin(x) на cos(x) и обработка случая, когда cos(x) близок к нулю;
- для CotFunction и CscFunction дополнительно проверялись случаи, когда знаменатель равен нулю или слишком мал;

- для CosFunction проверялось, что модуль корректно обращается к зависимой функции с преобразованным аргументом.

Выбор такого способа тестирования позволяет локализовать ошибки на уровне отдельного класса и повысить точность анализа результатов тестирования.

3. Модульные тесты итоговой системы функций

После тестирования отдельных модулей были разработаны тесты для SystemFunction. Данный класс представляет собой верхний уровень системы и содержит две ветви вычислений:

- тригонометрическую ветвь при x <= 0;
- логарифмическую ветвь при x > 0.

При тестировании SystemFunction использовались заглушки для всех зависимых функций. Это позволило отдельно проверить:

- корректность выбора ветви вычисления в зависимости от значения аргумента;
- корректность вычисления значения по формуле в каждой ветви;
- корректную обработку ситуаций, когда один из зависимых модулей возвращает NaN;
- корректную обработку случаев, когда знаменатель выражения оказывается слишком мал.

Такой уровень тестирования необходим, поскольку даже при корректной работе отдельных модулей итоговая система может содержать ошибки в логике композиции и ветвления.

4. Интеграционные тесты

После завершения модульного тестирования было выполнено интеграционное тестирование по стратегии **снизу вверх**. Данная стратегия была выбрана потому, что архитектура приложения явно содержит нижележащие базовые модули и модули верхнего уровня, зависящие от них.

Интеграция выполнялась в следующем порядке:

1. интеграция логарифмической подсистемы:
   LnFunction → Log3Function, Log10Function;
2. интеграция тригонометрической подсистемы:
   SinFunction → CosFunction → TanFunction, CotFunction, CscFunction;
3. интеграция полной системы в классе SystemFunction.

Такой порядок позволяет сначала убедиться в корректной работе нижних уровней, а затем последовательно подключать более сложные модули. В результате упрощается поиск причин ошибки: если интеграционный тест не проходит, можно достаточно точно определить уровень, на котором возникла проблема.

5. Обоснование выбранного тестового покрытия

Выбранное тестовое покрытие является достаточным для данной лабораторной работы по следующим причинам:

- покрыты базовые функции, реализованные через разложение в ряд;
- покрыты производные функции, зависящие от базовых;
- покрыта итоговая система функций как на уровне изолированных тестов, так и на уровне интеграции;
- проверены как корректные значения, так и недопустимые области;
- проверены граничные случаи, связанные с нулевыми и близкими к нулю знаменателями;
- покрыты оба направления интеграции, предусмотренные архитектурой приложения: тригонометрическое и логарифмическое.

Таким образом, тестовое покрытие охватывает основные сценарии использования приложения, ошибочные входные данные, граничные условия и особенности вычисления составной функции. Это позволяет считать разработанный набор тестов обоснованным и достаточным для проверки корректности работы системы.

Графики, построенные csv-выгрузкам, полученным в процессе интеграции приложения

В процессе интеграции приложения были получены CSV-выгрузки значений отдельных модулей и итоговой системы функций. На их основе были построены графики, отражающие работу логарифмической подсистемы, тригонометрической подсистемы и полной системы функций после завершения интеграции.

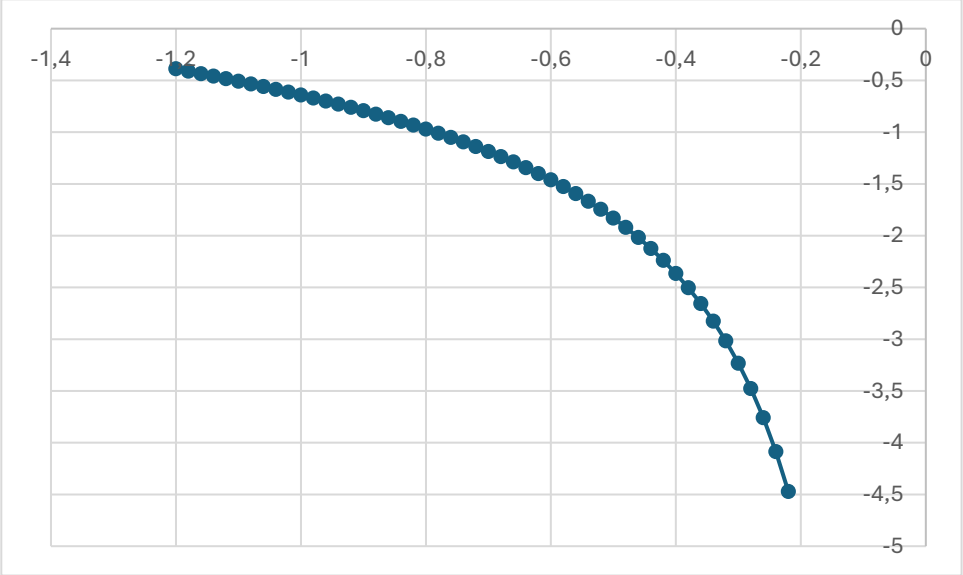


Рис. 3. График функции cot(x), построенный по CSV-выгрузке.

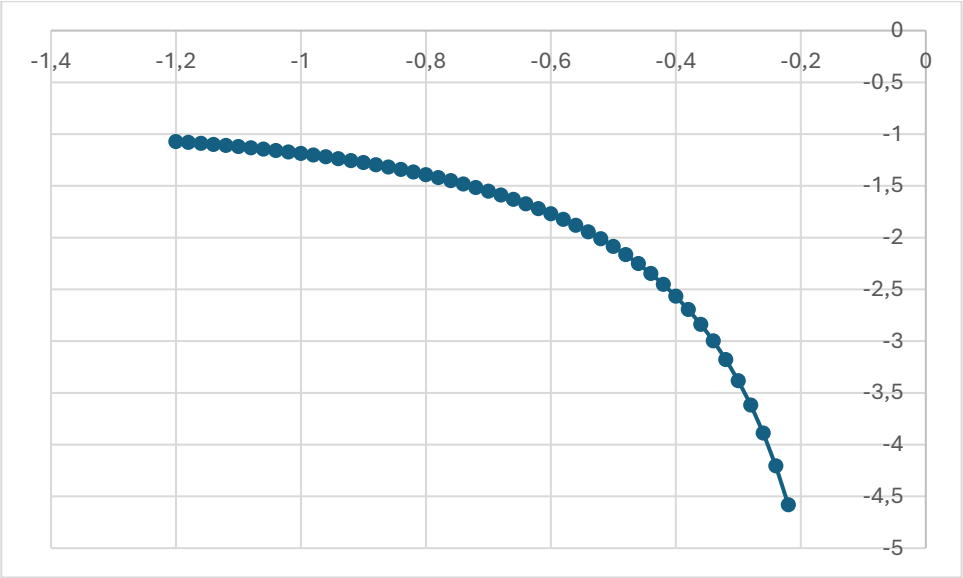


Рис. 4. График функции csc(x), построенный по CSV-выгрузке.

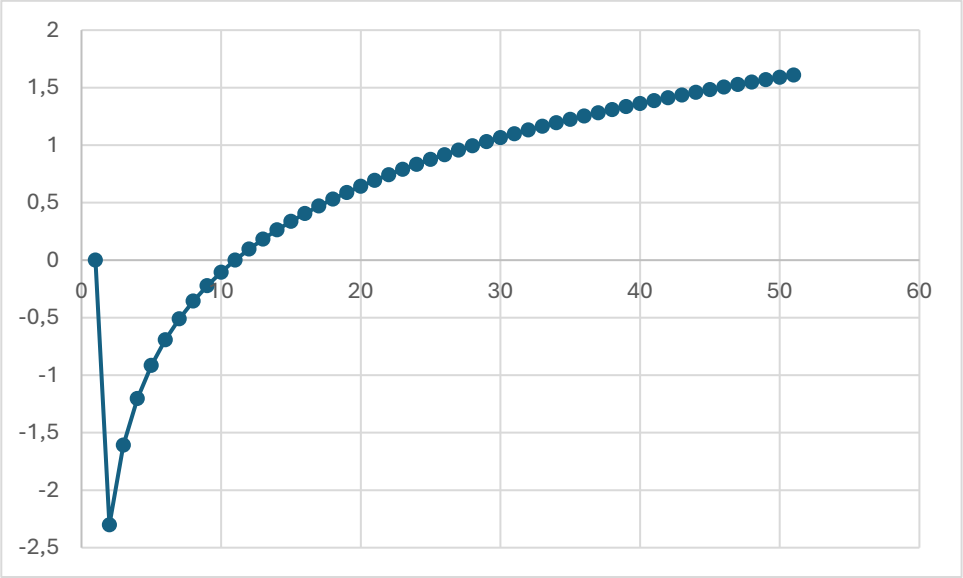


Рис. 5. График функции ln(x), построенный по CSV-выгрузке.

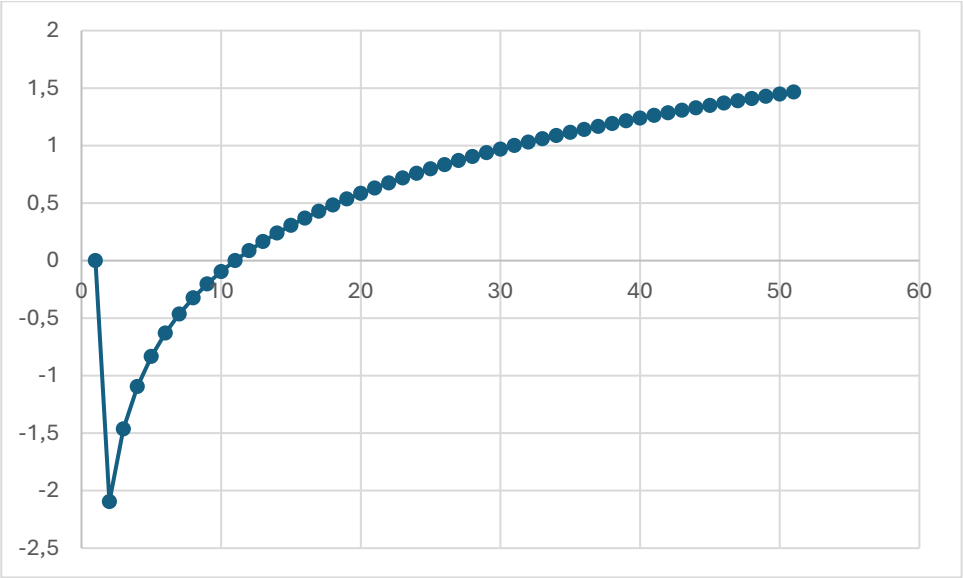


Рис. 6. График функции $log_3(x)$, построенный по CSV-выгрузке.

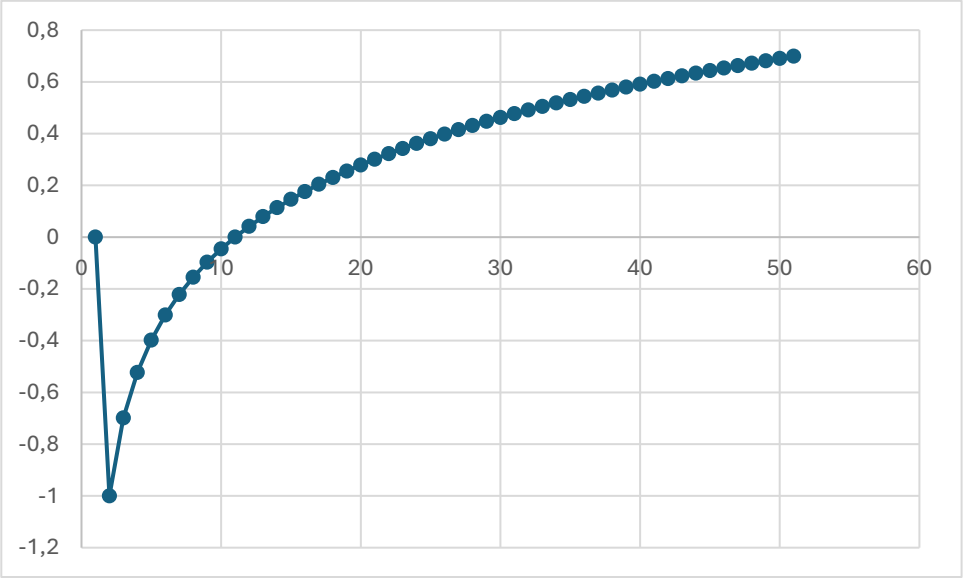


Рис. 7. График функции $log_{10}(x)$, построенный по CSV-выгрузке.

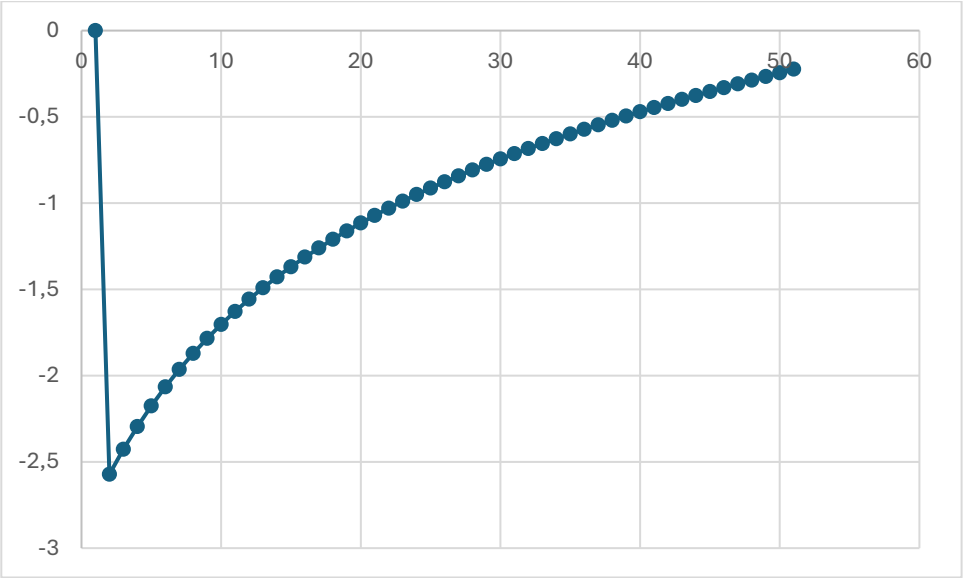


Рис. 8. График функции $\tan(x)$, построенный по CSV-выгрузке.

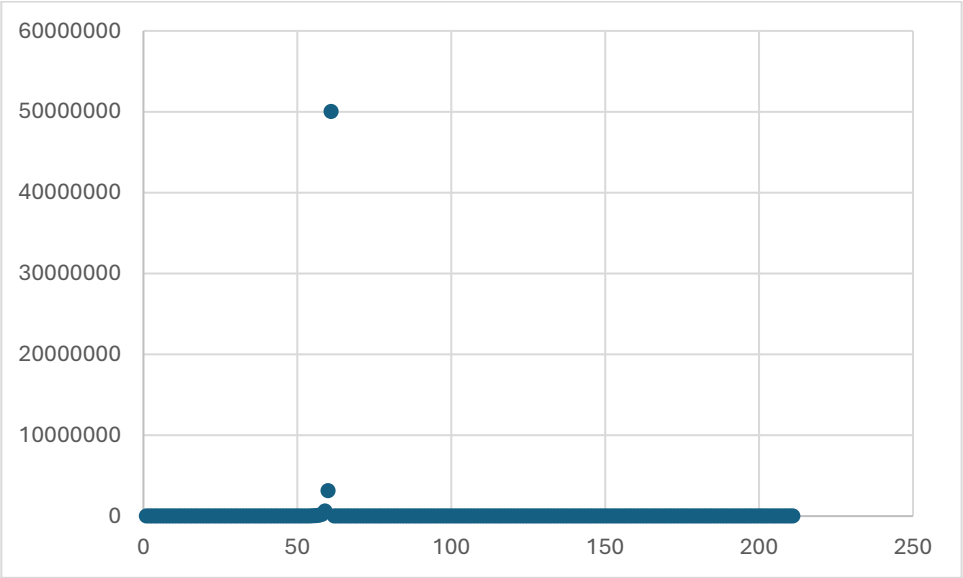


Рис. 9. График итоговой системы функций, построенный по CSV-выгрузке.

Вывод по работе

В ходе выполнения лабораторной работы было разработано приложение для вычисления системы функций в соответствии с заданным вариантом. Базовые функции sin(x) и ln(x) были реализованы с использованием разложения в ряд с заданной точностью, а остальные тригонометрические и логарифмические функции были построены на их основе.

В процессе работы была спроектирована модульная структура приложения, включающая базовые и производные функции, итоговую систему функций, табличные заглушки и механизм CSV-выгрузки результатов. Для проверки корректности работы программы было реализовано многоуровневое тестовое покрытие, включающее модульные и интеграционные тесты.

Интеграционное тестирование проводилось по стратегии снизу вверх: сначала проверялись базовые модули, затем производные подсистемы, после чего была выполнена интеграция всей системы функций. Дополнительно корректность работы приложения подтверждалась построением графиков по CSV-выгрузкам, полученным на различных этапах интеграции.

Таким образом, поставленная цель работы была достигнута: разработанное приложение корректно вычисляет заданную систему функций, поддерживает поэтапную интеграцию модулей и позволяет выполнять её тестирование в соответствии с требованиями лабораторной работы.